

DOSSIER DE PROJET

Titre Professionnel Développeur Web et Web Mobile

"Trouve ton artisan"

Sarah Brahimi

Centre Européen de Formation (CEF)

Promotion DWWM 2026

Sommaire

Introduction	3
1. Liste des compétences mises en œuvre	4
2. Expression des besoins	5
3. Environnement technique	6
4. Réalisations qui prouvent chaque compétence	9
4.1 Maquetter les interfaces utilisateur	9
4.2 Réaliser les interfaces statiques	11
4.3 Développer la partie dynamique des interfaces	16
4.4 Mettre en place une base de données relationnelle	19
4.5 Accès aux données SQL	22
4.6 Composants métier côté serveur	24
4.7 Sécurité	26
4.8 Sémantique HTML et SEO	28
4.9 Accessibilité et inclusion	29
Méthodologie et organisation du travail	30
Schéma de navigation	31
Jeu d'essai représentatif	32
Stratégie de validation	34
Veille technologique et sécurité	34
Bilan technique global	36
5. Difficultés rencontrées et solutions	36
Conclusion	40
Annexes	42
Remerciements	48

Introduction

Je m'appelle Sarah Brahimi et je suis actuellement en formation au Titre Professionnel Développeur Web et Web Mobile au Centre Européen de Formation (CEF). Avant d'entamer cette reconversion, j'ai travaillé dans des domaines variés (vente, télécommunications, animation d'activités pour enfants), sans lien direct avec le développement web. C'est un intérêt croissant pour la création de sites et d'applications qui m'a conduite à me former à ce métier.

Ce dossier présente le projet réalisé au cours de ma formation pour couvrir l'ensemble des six compétences obligatoires du Référentiel Emploi Activités Compétences (REAC) du Titre Professionnel : "Trouve ton artisan", une application fullstack de mise en relation entre particuliers et artisans, complétée d'un espace d'administration (back-office) permettant de gérer les artisans, catégories et spécialités de manière sécurisée.

Ce document suit le plan imposé par le Référentiel d'Évaluation (RE) pour les projets réalisés en centre de formation : liste des compétences mises en œuvre, expression des besoins, environnement technique, puis présentation détaillée des réalisations prouvant chaque compétence, illustrées de captures d'écran et d'extraits de code.

1. Liste des compétences mises en œuvre

1.1 Présentation générale du projet

Ce dossier s'appuie sur le projet "Trouve ton artisan", réalisé au cours de ma formation au Titre Professionnel Développeur Web et Web Mobile au Centre Européen de Formation.

"Trouve ton artisan" est une application fullstack complète, composée de deux volets complémentaires. Le premier est un site public : une plateforme de mise en relation entre particuliers et artisans de la région Auvergne-Rhône-Alpes, organisée par catégories et spécialités de métiers (bâtiment, artisanat d'art, alimentation, services). Le second est un espace d'administration (back-office), accessible uniquement à l'administrateur après authentification, permettant de créer, modifier et supprimer les artisans, catégories et spécialités présentés sur le site public.

Ce projet couvre ainsi l'intégralité de la chaîne de développement front-end (conception, intégration, dynamisation) côté public, la mise en place et l'exploitation d'une base de données relationnelle MySQL, ainsi que le développement de composants métier sécurisés côté serveur pour l'ensemble des opérations de lecture et d'écriture.

La combinaison de ces deux volets (site public et back-office) permet de couvrir l'intégralité des six compétences obligatoires du référentiel, réparties entre les deux activités types du Titre Professionnel.

1.2 Mise en œuvre des six compétences dans le projet

Le tableau ci-dessous récapitule les six compétences attendues et leur mise en œuvre respective :

Compétence	Périmètre
Maquetter des interfaces utilisateur web ou web mobile	Site public (5 écrans maquetés sur Figma)
Réaliser des interfaces utilisateur statiques web ou web mobile	Site public
Développer la partie dynamique des interfaces utilisateur	Site public et back-office
Mettre en place une base de données relationnelle	Base MySQL commune (site public + back-office)
Développer des composants d'accès aux données SQL	Site public et back-office (Sequelize)
Développer des composants métier côté serveur	Site public et back-office

2. Expression des besoins

2.1 Trouve ton artisan

Contexte

Ce projet répond à un besoin métier concret : faciliter la mise en relation entre des particuliers cherchant un artisan qualifié et des professionnels locaux, dans un secteur où la recherche se fait encore souvent par bouche-à-oreille ou via des annuaires peu structurés. L'application vise à centraliser cette recherche par catégorie et spécialité de métier, avec une identité visuelle propre à la région Auvergne-Rhône-Alpes.

Objectifs fonctionnels

- Permettre à un visiteur de parcourir les artisans par catégorie (Bâtiment, Services, Fabrication, Alimentation) puis par spécialité
- Mettre en avant une sélection d'artisans (« artisans du mois ») sur la page d'accueil
- Permettre la consultation d'une fiche détaillée par artisan (note, ville, description, site web)
- Offrir une fonctionnalité de recherche par nom
- Permettre le contact direct d'un artisan via un formulaire, avec envoi réel d'un email de notification
- Offrir à l'administrateur un espace dédié (back-office) pour créer, modifier et supprimer les artisans, catégories et spécialités, sans intervention manuelle en base de données

Objectifs techniques

- Développer une interface fidèle à une maquette Figma définie en amont, avec une palette de couleurs imposée
- Structurer les données dans une base relationnelle normalisée (catégorie → spécialité → artisan)
- Déployer l'ensemble (front-end, API, base de données) sur des services d'hébergement distincts, à la manière d'une mise en production réelle

Limites fixées

- Pas de système de compte côté visiteur : le site public reste en consultation libre, sans création de compte ; seul un compte administrateur unique existe, pour la gestion du contenu
- Pas de prise de rendez-vous automatisée : le contact se fait via formulaire, la mise en relation reste manuelle, sans confirmation en temps réel
- Utilisation de données de démonstration (artisans fictifs), le projet n'étant pas lié à un commanditaire réel

Public cible et cas d'usage

Le site s'adresse à deux profils d'utilisateurs distincts, bien que le projet ne mette en œuvre qu'un seul type d'interface (le site public, sans back-office de gestion pour les artisans eux-mêmes) :

- Les particuliers de la région Auvergne-Rhône-Alpes à la recherche d'un artisan qualifié pour un besoin ponctuel (travaux, réparation, service à domicile) : ce sont les utilisateurs principaux du site, qui naviguent par catégorie ou effectuent une recherche directe par nom

- Les artisans eux-mêmes, dont les informations (coordonnées, spécialité, ville, site web) sont présentées sur le site, mais qui n'ont pas d'accès direct à une interface de gestion de leur fiche dans le cadre de ce devoir — cette évolution est envisageable pour une version future du projet

Le cas d'usage principal peut se résumer ainsi : un particulier a besoin d'un plombier dans sa ville, il consulte la catégorie "Bâtiment", filtre visuellement les artisans par spécialité, consulte la fiche de celui qui lui semble le plus adapté (note, ville, description), puis le contacte directement via le formulaire présent sur cette fiche.

Contraintes de réalisation

Ce projet a été réalisé dans un cadre de formation, avec plusieurs contraintes propres à ce contexte :

- Délai : le projet devait être finalisé dans le temps imparti par le calendrier de formation, ce qui a orienté certains choix vers des solutions simples et rapides à mettre en œuvre plutôt que vers des architectures plus complexes (par exemple, EmailJS plutôt qu'un serveur mail dédié)
- Budget : l'ensemble des services utilisés (hébergement, base de données, envoi d'emails) devait rester gratuit, ce qui a orienté le choix des prestataires (Netlify, Render, Aiven, EmailJS) et explique certaines limites rencontrées, détaillées dans la partie « Difficultés rencontrées et solutions »
- Cahier des charges imposé : une partie des choix de conception (palette de couleurs, écrans à réaliser, structure de navigation) était définie en amont par le brief du devoir, comme détaillé en partie 4.1

2.2 Livrables attendus et livrés

Le livrable final ne se limite pas à l'interface visible par le visiteur. Il correspond à un ensemble cohérent composé du site public, de son API, de la base de données et de l'espace d'administration.

- Site web public : interface responsive permettant de rechercher et consulter les artisans et de les contacter.
- API REST : application Node.js/Express permettant au front-end d'accéder aux données MySQL.
- Base de données MySQL : catégories, spécialités et artisans, avec leurs relations et clés étrangères.
- Back-office administrateur : interface protégée permettant l'ajout, la modification et la suppression des artisans, catégories et spécialités, ainsi que le changement du mot de passe administrateur.
- Code source : dépôts GitHub séparés pour le front-end et l'API, afin de conserver et transmettre le projet.

Le résultat livré est donc une application full-stack utilisable sur desktop, tablette et mobile, accompagnée d'un outil d'administration permettant de maintenir les données sans intervention directe dans la base.

3. Environnement technique

3.1 Trouve ton artisan

Front-end

Technologie	Usage
React	Bibliothèque JavaScript utilisée pour construire l'interface en composants réutilisables et gérer l'état de l'application de façon réactive
React Router	Gestion du routing côté client, permettant une navigation fluide entre les pages sans rechargement complet
Bootstrap	Framework CSS utilisé comme base de grille responsive et de composants (formulaires, boutons, navigation)
Sass	Préprocesseur CSS permettant de structurer les styles en variables et fichiers modulaires, et de personnaliser Bootstrap via ses propres variables
Axios	Bibliothèque de requêtes HTTP utilisée pour communiquer avec l'API back-end

Back-end

Technologie	Usage
Node.js	Environnement d'exécution JavaScript côté serveur
Express	Framework minimaliste utilisé pour construire l'API REST (routes, middlewares)
Sequelize	ORM (Object-Relational Mapping) permettant de manipuler la base MySQL via des modèles JavaScript plutôt que du SQL brut

Outils de conception et de versioning

Technologie	Usage
Figma	Conception des maquettes d'interface
Mocodo	Génération du schéma conceptuel de données (MCD)
Git / GitHub	Gestion de versions et hébergement du code source
VS Code	Environnement de développement, avec l'extension Prettier pour le formatage automatique du code

Base de données et hébergement

3.2 Architecture générale et flux de données

L'application « Trouve ton artisan » repose sur une architecture client-serveur qui sépare l'interface utilisateur, l'API et la base de données. Cette organisation permet de distinguer les responsabilités de chaque partie de l'application et facilite les évolutions futures.

Côté public, l'utilisateur interagit avec une interface développée en React. Les composants React déclenchent des requêtes HTTP vers l'API au moyen d'Axios. Le serveur Node.js/Express reçoit ces requêtes, applique la logique nécessaire et utilise Sequelize pour accéder à la base MySQL. Les données retournées par l'API sont ensuite exploitées par React pour mettre à jour l'affichage. Le même principe est utilisé pour le BackOffice. La différence principale réside dans le contrôle d'accès : les opérations d'écriture sont protégées par une authentification JWT. Le serveur vérifie le jeton avant d'autoriser les opérations de création, modification ou suppression.

Flux d'une requête

Site public : Utilisateur → React → Axios → API Express → Sequelize → MySQL → API → React → affichage des données.

BackOffice : Administrateur → React → authentification JWT → route protégée Express → controller → service → Sequelize → MySQL.

3.3 Installation et configuration de l'environnement

L'environnement de développement a été configuré autour de Visual Studio Code, Node.js et npm. Le projet est versionné avec Git et hébergé sur GitHub afin de conserver l'historique des modifications et de disposer d'un point de sauvegarde du code source. Les dépendances du front-end et du back-end sont déclarées dans leurs fichiers package.json respectifs afin de pouvoir reconstituer l'environnement du projet.

La configuration sensible, notamment les informations de connexion à la base de données et les paramètres nécessaires aux services externes, est séparée du code source grâce aux variables d'environnement. Le fichier .env n'est pas versionné, conformément à la configuration du .gitignore. Cette organisation distingue les paramètres propres à l'environnement local de ceux utilisés lors du déploiement.

Élément	Choix	Justification
Base de données	MySQL, hébergée sur Aiven	Base relationnelle managée, offre gratuite pérenne (après une première tentative sur Railway devenue payante)
Hébergement front-end	Netlify	Déploiement automatique à chaque push Git, gratuit pour un projet de cette taille
Hébergement API	Render	Hébergement Node.js gratuit avec déploiement continu depuis GitHub
Envoi d'emails	EmailJS	Solution permettant d'envoyer un email réel depuis le front-end sans back-end mail dédié

4. Réalisations qui prouvent chaque compétence

4.1 Maquetter des interfaces utilisateur web ou web mobile

Cadre du projet

Le projet "Trouve ton artisan" étant réalisé dans le cadre d'un devoir de formation, plusieurs éléments de conception m'ont été imposés par le cahier des charges :

- La palette de couleurs imposée :
 - \$primary — #0074c7 (bleu principal)
 - \$secondary — #00497c (bleu foncé)
 - \$dark — #384050 (gris foncé, texte)
 - \$light — #f1f8fc (fond clair)
 - \$danger — #cd2c2e (rouge, erreurs)
 - \$success — #82b864 (vert, succès)
- Les 5 écrans à concevoir : page d'accueil, liste des artisans par catégorie, fiche détail d'un artisan, page mentions légales, page 404
- La structure générale de navigation : un menu de catégories en en-tête, une barre de recherche, un pied de page fixe avec les informations légales

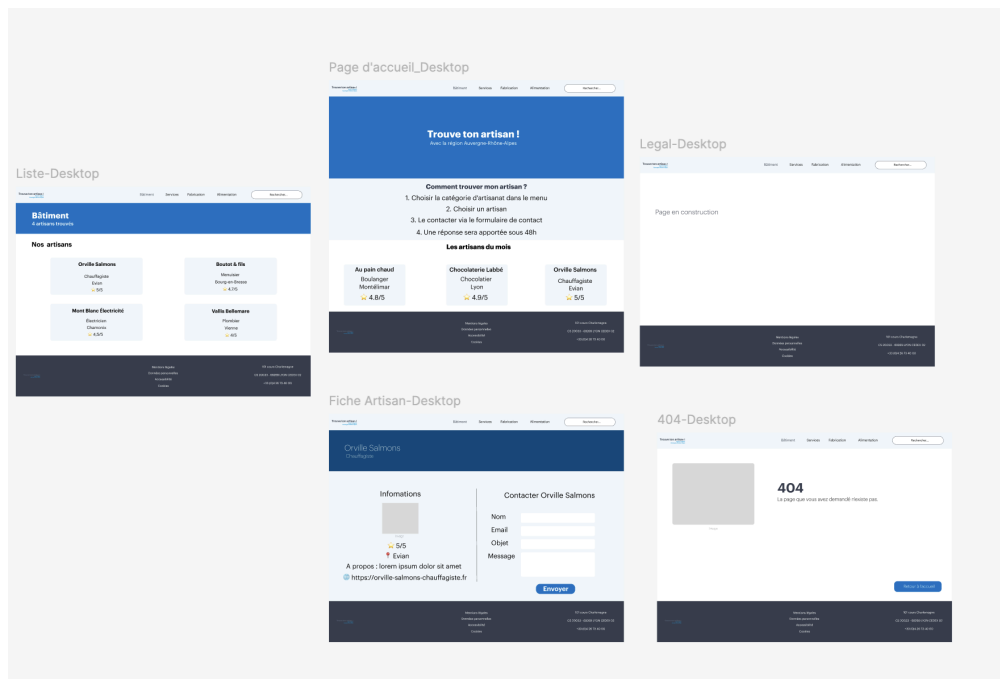
Démarche de conception

Sur la base de ces contraintes, j'ai réalisé les maquettes sur Figma en respectant une logique de conception centrée sur la simplicité de navigation :

- Page d'accueil : structurée en 3 blocs verticaux — un bandeau d'introduction avec le message clé du site, un guide en 4 étapes expliquant le fonctionnement ("Comment trouver mon artisan ?"), et une mise en avant des artisans du mois
- Page catégorie : un bandeau de titre reprenant la couleur primaire, suivi d'une grille de cartes présentant chaque artisan de la catégorie sélectionnée
- Fiche artisan : une mise en page en deux colonnes, séparant les informations de l'artisan (à gauche) du formulaire de contact (à droite), pour permettre à l'utilisateur de consulter et d'agir sans changer de page
- Pages secondaires (mentions légales, 404) : conservent l'identité visuelle du site (en-tête, pied de page) tout en restant volontairement sobres dans leur contenu

Marge de liberté créative

Dans le cadre de ces contraintes imposées, j'ai conservé une marge de liberté sur la mise en page, l'agencement des éléments et le style visuel : espacements, arrondi des composants (boutons, champ de recherche en forme de pilule), et hiérarchie visuelle entre les différents blocs de chaque page. Bien que la palette de couleurs elle-même ait été imposée, le choix de l'endroit où appliquer chaque couleur restait libre : par exemple, j'aurais pu utiliser le bleu primaire ou le blanc pour la barre de navigation, ce choix d'application ne faisait pas partie des contraintes du cahier des charges.



Maquettes Figma de "Trouve ton artisan" — pages Liste, Accueil, Legal, Fiche Artisan, 404

4.2 Réaliser des interfaces utilisateur statiques web ou web mobile

Architecture des composants

À partir des maquettes Figma, j'ai développé l'intégration statique du site avec React, en découpant chaque page en composants réutilisables plutôt qu'en blocs de code monolithiques :

- Header : menu de navigation, logo, barre de recherche — présent sur toutes les pages
- Footer : informations légales et de contact — présent sur toutes les pages
- Cartes artisan : composant réutilisé aussi bien sur la page d'accueil ("artisans du mois") que sur les pages catégorie, évitant la duplication de code
- Pages (Accueil, Catégorie, FicheArtisan, Legal, NotFound) : chacune assemble les composants nécessaires à sa mise en page spécifique

Cette approche par composants respecte un principe fondamental de React : chaque élément d'interface récurrent n'est écrit qu'une seule fois, puis réutilisé et paramétré via des props à chaque endroit où il apparaît.

Mise en forme avec Sass et Bootstrap

Le style visuel s'appuie sur Bootstrap comme base de grille responsive (système de colonnes `row/col-md-*`) et de composants prêts à l'emploi (formulaires, boutons, cartes). Cette base a été personnalisée pour respecter l'identité visuelle imposée :

- Le fichier `main.scss` centralise les variables Sass utilisées par le projet (`$primary`, `$secondary`, `$dark`, `$light`, `$danger`, `$success`). Elles permettent de conserver une palette cohérente dans les styles personnalisés.
- Des styles Sass complémentaires peuvent être organisés par composant. Le projet contient notamment `Header.scss`, même si les styles effectivement chargés au démarrage sont principalement centralisés dans `main.scss`.

Exemple : le composant Header

```

1  import React, { useState, useEffect } from "react";
2  import { Link, useNavigate } from "react-router-dom";
3  import axios from "axios";
4
5  const API_URL = process.env.REACT_APP_API_URL || "http://localhost:4000/api";
6
7  function Header() {
8    const [categories, setCategories] = useState([]);
9    const [search, setSearch] = useState("");
10   const navigate = useNavigate();
11
12   useEffect(() => {
13     axios
14       .get(`${API_URL}/categories`)
15       .then((res) => setCategories(Array.isArray(res.data) ? res.data : []))
16       .catch((err) => console.error(err));
17   }, []);
18
19   const handleSearch = (e) => {
20     e.preventDefault();
21     if (search.trim()) {
22       navigate(`/search/${search}`);
23     }
24   };
25
26   return (
27     <header className="navbar navbar-expand-lg px-0">
28       <div className="container">
29         <Link className="navbar-brand me-4" to="/">
30           
31         </Link>
32         <button
33           className="navbar-toggler"

```

Le composant Header combine une structure Bootstrap standard (navbar, container) avec des éléments de navigation React et un formulaire de recherche. Son comportement responsive repose principalement sur les classes Bootstrap utilisées dans le composant.

```

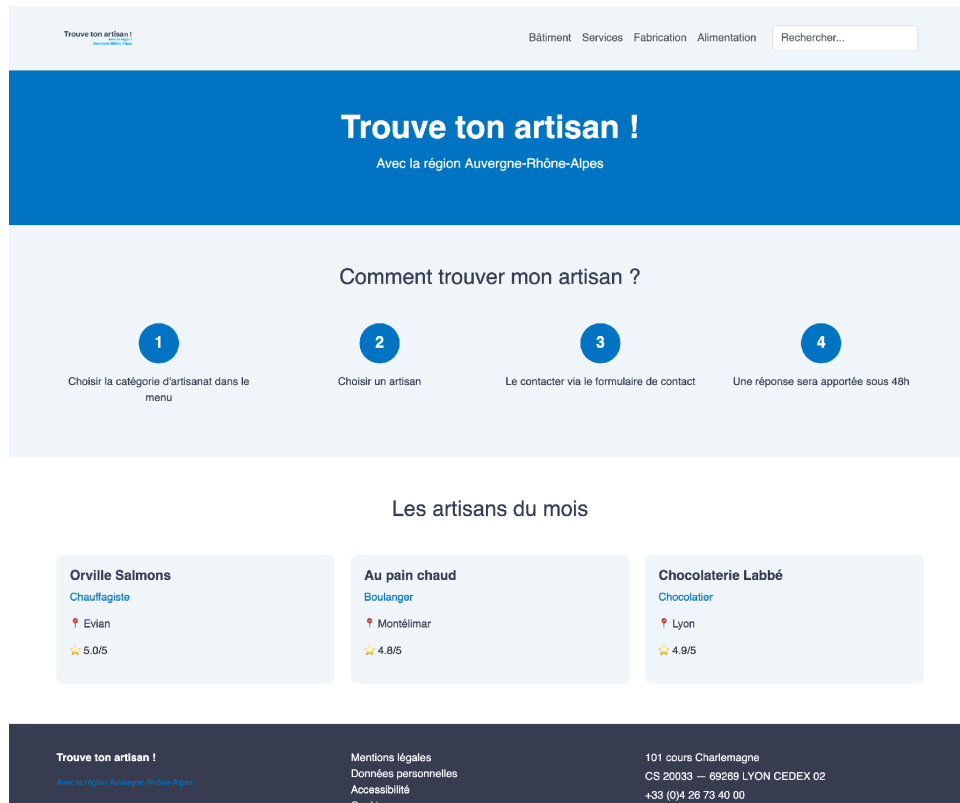
1  @import "../styles/variables"; // adapte le chemin vers mon fichier de variables
2
3  .custom-header {
4    background-color: $light;
5
6    .navbar-brand img {
7      display: block;
8    }
9
10   .nav-link {
11     color: $dark;
12     font-weight: 500;
13     margin: 0 12px;
14     transition: color 0.2s;
15
16     &:hover {
17       color: $primary;
18     }
19   }
20
21   .search-input {
22     border-radius: 50px;
23     border: 1px solid $secondary;
24     padding: 6px 18px;
25
26     &:focus {
27       outline: none;
28       border-color: $primary;
29       box-shadow: none;
30     }
31   }
32 }
33

```

Ce fichier Sass illustre la personnalisation de Bootstrap évoquée plus haut : les couleurs proviennent des variables globales (\$light, \$dark, \$primary, \$secondary), tandis que des propriétés spécifiques (border-radius: 50px pour la barre de recherche en forme de pilule, transition sur le survol des liens) viennent enrichir le rendu par défaut de Bootstrap sans le dénaturer.

Responsive design

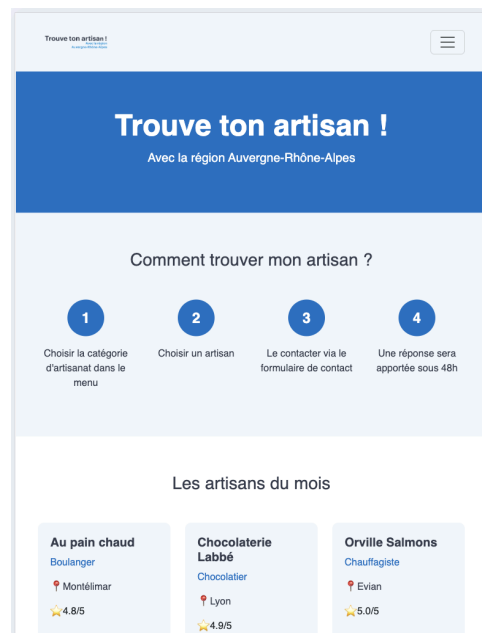
L'ensemble de l'interface a été conçu pour s'adapter à trois types d'écrans, grâce au système de grille Bootstrap : desktop (affichage en plusieurs colonnes), tablette (réduction du nombre de colonnes), et mobile (menu de navigation transformé en menu déroulant grâce au composant navbar-toggler de Bootstrap, cartes empilées verticalement).



Page d'accueil du site "Trouve ton artisan"

Illustration du responsive (desktop, tablette et mobile)

Les captures ci-dessous permettent de comparer directement les trois formats livrés : desktop, tablette et mobile.



Page d'accueil en version tablette

Sur tablette, la disposition se rapproche de la version desktop : le menu bascule déjà en menu « hamburger » (icône à trois barres, via le composant navbar-toggler de Bootstrap) pour préserver la

lisibilité sur une largeur réduite, mais l'espacement et la taille des blocs restent proches de la version desktop.



Page d'accueil en version mobile

Sur mobile, le menu hamburger est conservé, le titre principal repasse automatiquement sur deux lignes, et les étapes du bloc « Comment trouver mon artisan ? » s'empilent verticalement au lieu de s'aligner en quatre colonnes. Cette adaptation repose entièrement sur le système de grille responsive de Bootstrap (col-md-*), qui ajuste automatiquement le nombre de colonnes affichées selon la largeur de l'écran, sans media queries personnalisées.

4.3 Développer la partie dynamique des interfaces utilisateur web ou web mobile

Récupération et affichage des données

La dynamique de l'application repose principalement sur `useEffect` et `useState`. Par exemple, le composant `Header` récupère les catégories via `Axios`, tandis que la page d'accueil récupère les artisans mis en avant depuis l'endpoint `/artisans/top`. Les résultats sont ensuite stockés dans l'état `React` et parcourus avec `map()`.

Cette approche permet à l'interface de rester synchronisée avec le contenu réel de la base de données : toute modification des données côté back-end (ajout d'un artisan, changement de catégorie) se répercute automatiquement sur le site sans intervention manuelle sur le code front-end.

Routing dynamique

`React Router` gère la navigation entre les pages sans rechargement complet du navigateur. Deux routes illustrent particulièrement la dimension dynamique du site :

- `/categorie/:id` : le composant `Categorie` récupère l'identifiant de la catégorie depuis l'URL via le hook `useParams`, puis l'utilise pour filtrer les artisans à afficher via l'API
- `/artisan/:id` : même principe pour afficher la fiche détaillée d'un artisan spécifique

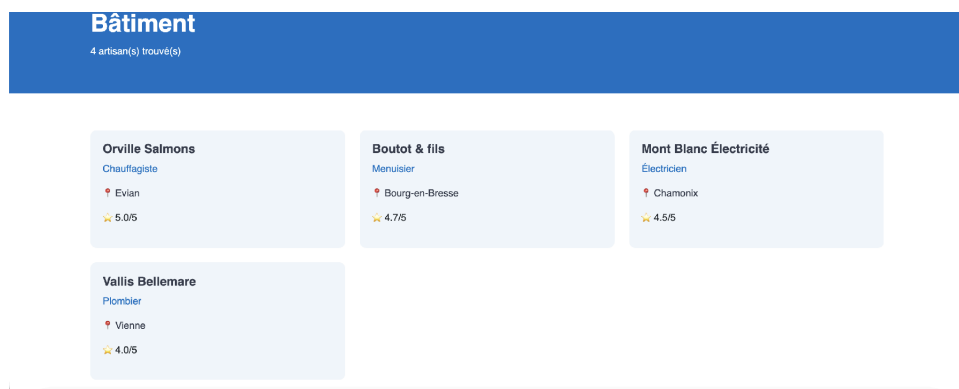
Cette architecture évite de coder une page par artisan : une seule page-modèle s'adapte dynamiquement en fonction du paramètre d'URL.

Formulaire de contact interactif

Le formulaire de la fiche artisan illustre la gestion d'un état de formulaire contrôlé en `React` : chaque champ (nom, email, objet, message) est relié à une propriété de l'état du composant, mise à jour en temps réel à chaque frappe (`onChange`). À la soumission, une fonction asynchrone envoie ces données à l'API `EmailJS`, puis met à jour l'affichage selon le résultat : un message de confirmation en cas de succès, une alerte en cas d'échec — le tout géré par un affichage conditionnel basé sur l'état du composant.

Recherche dynamique

Un champ de recherche présent dans l'en-tête permet de rediriger l'utilisateur, à la soumission, vers une route de résultats qui interroge l'API avec le terme saisi et affiche les artisans correspondants.



Page catégorie "Bâtiment" — liste des artisans chargée dynamiquement depuis l'API

Orville Salmons
Chauffagiste

Photo

★ 5.0/5
📍 Evian

À propos
Chauffagiste expérimenté, interventions rapides et soignées.
<https://orville-salmons-chauffagiste.fr>

Contacter Orville Salmons

Nom

Email

Objet

Message

Envoyer

Fiche artisan avec formulaire de contact interactif

Interface d'administration (back-office)

L'espace d'administration, accessible via la route /admin, met également en œuvre la dimension dynamique des interfaces : formulaires contrôlés pour créer et modifier les artisans, catégories et spécialités, mise à jour immédiate du tableau après chaque action, et protection de l'ensemble des pages d'administration derrière une vérification d'authentification.

Trouve ton artisan !
[Apprends à connaître les artisans](#)

Alimentation Bâtiment Fabrication Services Rechercher...

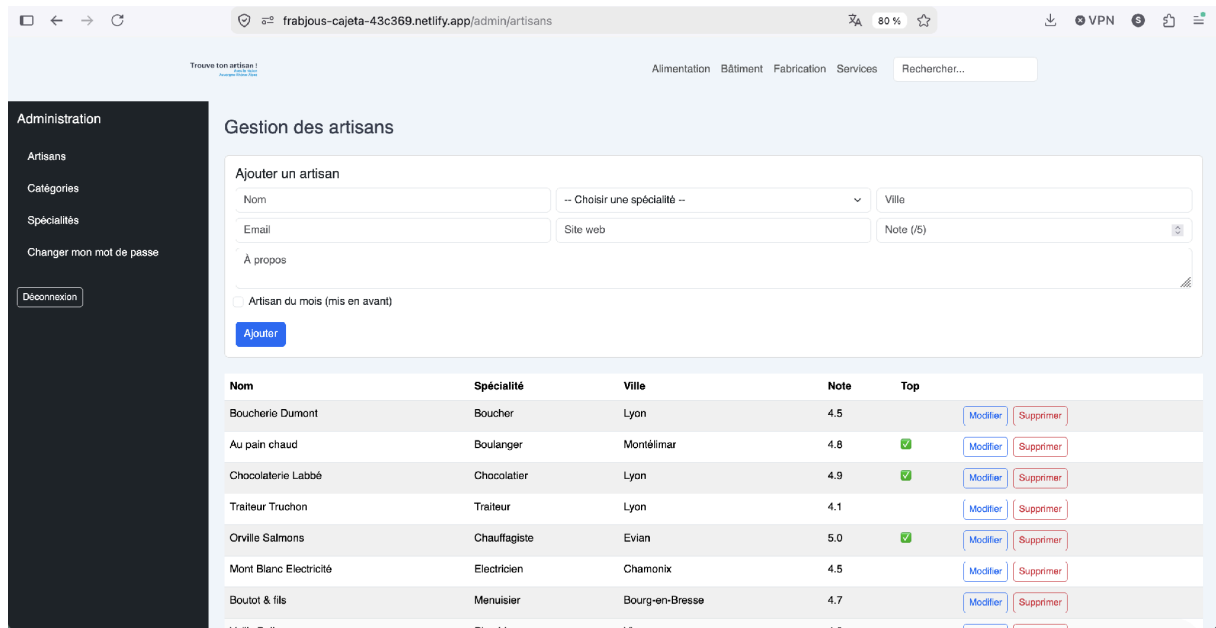
Espace administrateur

Email
sarah.brahimi03@gmail.com

Mot de passe

Se connecter

Page de connexion de l'espace d'administration



Gestion des artisans dans le back-office (ajout, modification, suppression)

L'accès à ces pages est conditionné à la présence d'un jeton JWT valide, vérifiée par un composant dédié avant l'affichage de tout contenu sensible :

```

6 function RequireAdminAuth({ children }) {
7   const token = localStorage.getItem("admin_token");
8
9   if (!token) {
10    return <Navigate to="/admin/login" replace />;
11  }
12
13  return children;
14 }
15
16 export default RequireAdminAuth;
17

```

Une instance Axios dédiée à l'administration injecte automatiquement ce jeton dans l'en-tête de chaque requête, via un intercepteur :

```

11 adminApi.interceptors.request.use((config) => {
12   const token = localStorage.getItem("admin_token");
13   if (token) {
14     config.headers.Authorization = `Bearer ${token}`;
15   }
16   return config;
17 });

```

Ce mécanisme évite de rajouter manuellement l'en-tête d'autorisation à chaque appel : toutes les requêtes vers les routes protégées de l'API (création, modification, suppression) passent automatiquement par cette instance et sont donc systématiquement authentifiées.

4.4 Mettre en place une base de données relationnelle

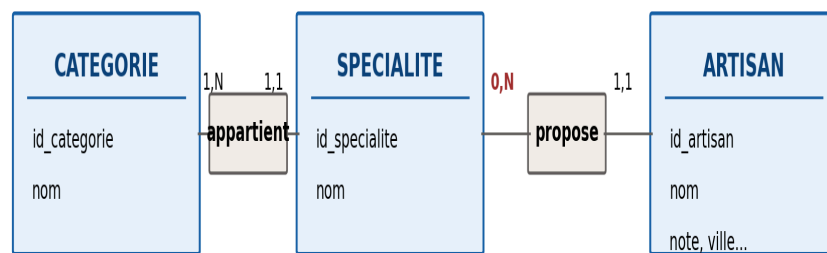
Conception du modèle

La base de données du projet "Trouve ton artisan" repose sur un schéma relationnel MySQL composé de 3 tables reliées en cascade, conçu selon la méthode Merise (MCD → MLD → MPD) :

- categorie (id_categorie, nom) : les grandes familles de métiers (Bâtiment, Services, Fabrication, Alimentation)
- specialite (id_specialite, nom, id_categorie) : les métiers précis, rattachés à une catégorie
- artisan (id_artisan, nom, note, ville, a_propos, email, site_web, top, id_specialite) : les fiches artisans, rattachées à une spécialité

Modèle Conceptuel de Données (MCD)

MCD corrigé — "Trouve ton artisan"
(cardinalité 0,N côté Spécialité pour la relation propose, en rouge = correction)



Modèle Conceptuel de Données (MCD) de "Trouve ton artisan", avec la cardinalité corrigée (0,N)

Script SQL de création

```

1 CREATE DATABASE IF NOT EXISTS trouve_ton_artisan;
2 USE trouve_ton_artisan;
3
4 CREATE TABLE categorie (
5     id_categorie INT AUTO_INCREMENT PRIMARY KEY,
6     nom VARCHAR(100) NOT NULL
7 );
8
9 CREATE TABLE specialite (
10    id_specialite INT AUTO_INCREMENT PRIMARY KEY,
11    nom VARCHAR(100) NOT NULL,
12    id_categorie INT NOT NULL,
13    FOREIGN KEY (id_categorie) REFERENCES categorie(id_categorie)
14 );
15
16 CREATE TABLE artisan (
17    id_artisan INT AUTO_INCREMENT PRIMARY KEY,
18    nom VARCHAR(150) NOT NULL,
19    note DECIMAL(2,1),
20    ville VARCHAR(100),
21    a_propos TEXT,
22    email VARCHAR(150),
23    site_web VARCHAR(255),
24    top BOOLEAN DEFAULT FALSE,
25    id_specialite INT NOT NULL,
26    FOREIGN KEY (id_specialite) REFERENCES specialite(id_specialite)
27 );

```

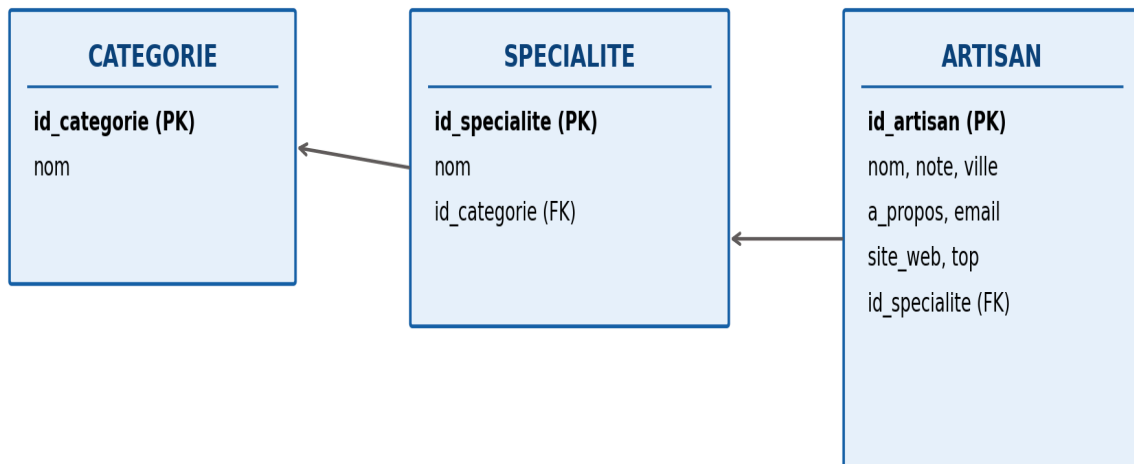
Intégrité référentielle

Chaque table possède une clé primaire auto-incrémentée (`id_categorie`, `id_specialite`, `id_artisan`), garantissant l'unicité de chaque enregistrement. Les relations entre tables sont assurées par des clés étrangères : `specialite.id_categorie` référence `categorie.id_categorie`, et `artisan.id_specialite` référence `specialite.id_specialite`. Cette contrainte empêche la création d'une spécialité rattachée à une catégorie inexistante, ou d'un artisan rattaché à une spécialité inexistante — garantissant ainsi la cohérence des données à tout moment.

Modèle Logique de Données (MLD)

Le MLD ci-dessous dérive du MCD selon les règles de passage de la méthode Merise, en mettant en évidence les clés primaires (soulignées) et les clés étrangères (FK) issues des relations du MCD :

MLD — "Trouve ton artisan"
 (clés étrangères : id_categorie sur SPECIALITE, id_specialite sur ARTISAN)



Modèle Logique de Données (MLD) de "Trouve ton artisan"

Chaque relation du MCD de type 0,N ou 1,N côté A et 1,1 côté B se traduit par la migration de la clé primaire de A en clé étrangère dans B. C'est ainsi que id_categorie migre dans la table specialite (relation appartient), et que id_specialite migre dans la table artisan (relation propose). Aucune des deux relations n'étant de type N,N, aucune table de jointure n'est nécessaire : le MLD compte exactement les trois mêmes tables que le MCD.

Hébergement

La base est hébergée sur Aiven, un service MySQL managé. Ce choix fait suite à une première tentative sur Railway, dont l'offre d'essai gratuite a expiré en cours de projet — cette migration est détaillée dans la partie « Difficultés rencontrées et solutions ».

4.5 Développer des composants d'accès aux données SQL

4.5.1 Flux d'accès aux données

L'accès aux données suit un flux précis entre l'interface React, l'API Express, les modèles Sequelize et la base MySQL. Cette séparation évite que le front-end communique directement avec la base de données et centralise les opérations de données dans le back-end.

Lorsqu'un visiteur consulte une catégorie, React envoie une requête GET vers l'API. Le serveur reçoit la demande, identifie la ressource à retourner et interroge le modèle Sequelize correspondant. Sequelize traduit l'opération en requête SQL adaptée à MySQL, puis transforme le résultat en objet exploitable par le code JavaScript.

Les associations entre les modèles permettent de récupérer les informations liées à un artisan, sa spécialité et sa catégorie. L'utilisation de l'option `include` permet de limiter les appels côté client et de construire une réponse contenant les données nécessaires à l'affichage.

Pour les opérations d'administration, le flux est complété par une vérification du jeton JWT avant l'accès aux méthodes d'écriture. Ainsi, la même base SQL peut être consultée par le site public tout en restant modifiable uniquement depuis les fonctionnalités autorisées du BackOffice.

Accès SQL — Trouve ton artisan

L'API back-end utilise Sequelize comme ORM (Object-Relational Mapping) pour interagir avec la base MySQL sans écrire de requêtes SQL brutes. Chaque table de la base est représentée par un modèle JavaScript définissant ses colonnes, leurs types et leurs contraintes :

```

1  const { DataTypes } = require("sequelize");
2  const sequelize = require("../config/database");
3
4  const Artisan = sequelize.define(
5    "Artisan",
6    {
7      id_artisan: {
8        type: DataTypes.INTEGER,
9        primaryKey: true,
10       autoIncrement: true,
11      },
12      nom: {
13        type: DataTypes.STRING(150),
14        allowNull: false,
15      },
16      note: {
17        type: DataTypes.DECIMAL(2, 1),
18      },
19      ville: {
20        type: DataTypes.STRING(100),
21      },
22      a_propos: {
23        type: DataTypes.TEXT,
24      },
25      email: {
26        type: DataTypes.STRING(150),
27      },
28      site_web: {
29        type: DataTypes.STRING(255),
30      },
31      top: {
32        type: DataTypes.BOOLEAN,
33        defaultValue: false,
34      },
35      id_specialite: {
36        type: DataTypes.INTEGER,
37        allowNull: false,
38        allowNull: false,
39      },
40      {
41        tableName: "artisan",
42        timestamps: false,
43      },
44    });
45
46 module.exports = Artisan;
47

```

Au-delà de la définition des colonnes, Sequelize permet de déclarer les associations entre modèles (relations `hasMany/belongsTo`), qui reproduisent les clés étrangères de la base au niveau du code JavaScript. C'est ce qui permet, dans les routes de l'API, d'utiliser l'option `include` pour récupérer un artisan avec sa spécialité et sa catégorie en une seule requête, sans avoir à écrire de jointure SQL manuelle.

4.5.2 Opérations DML via Sequelize

Le DML (Data Manipulation Language) correspond aux opérations qui manipulent les données déjà présentes dans les tables : lecture, insertion, modification et suppression. Dans ce projet, ces opérations sont réalisées avec Sequelize plutôt qu'avec des requêtes SQL écrites manuellement.

Lecture — SELECT : `findAll()` récupère plusieurs enregistrements et `findByPk()` récupère un enregistrement à partir de sa clé primaire.

Insertion — INSERT : `createArtisan()` utilise `Artisan.create(data)` pour ajouter un artisan.

Modification — UPDATE : `updateArtisan()` recherche d'abord l'artisan puis utilise `artisan.update(data)`.

Suppression — DELETE : `deleteArtisan()` recherche l'artisan puis utilise `artisan.destroy()`.

Ces méthodes constituent la couche de manipulation des données. Les controllers n'accèdent pas directement à MySQL : ils délèguent au service, qui utilise Sequelize. Cela répond à la distinction entre DML et DDL : le DDL décrit la structure des tables, tandis que le DML agit sur les enregistrements.

4.6 Développer des composants métier côté serveur

Logique métier — Trouve ton artisan

L'architecture actuelle de l'API respecte la séparation route → controller → service. Les routes déclarent les endpoints et délèguent le traitement aux controllers ; les controllers gèrent la requête/réponse HTTP et les contrôles d'entrée ; les services portent les opérations métier et les accès aux modèles Sequelize.

```
const router = express.Router();

// La route déclare l'endpoint et délègue au controller.
router.get("/:id", artisanController.getById);
router.get("/search/:nom", artisanController.search);
router.post("/", requireAuth, artisanController.create);
```

Pour la récupération d'un artisan, le controller reçoit l'identifiant présent dans l'URL et appelle le service correspondant. Il transforme ensuite le résultat en réponse HTTP, par exemple 404 si l'artisan n'existe pas. La recherche suit le même principe.

La logique métier n'est donc pas placée dans la déclaration de route. Les opérations de recherche, de récupération, de création, de modification et de suppression sont regroupées dans les services. Cette organisation rend les responsabilités plus lisibles.

L'API actuelle est organisée selon le pattern route → controller → service. Une route ne contient pas la logique de traitement : elle déclare l'URL, associe la méthode HTTP au controller et applique si nécessaire un middleware. Le controller traite le cycle requête/réponse et les validations simples, tandis que le service réalise les opérations métier et les accès Sequelize.

La route se limite à déclarer les endpoints, en protégeant les routes d'écriture par le middleware d'authentification (détaillé en partie 4.7) :


```

trouve-ton-artisan-api > routes > JS artisans.js > router
1  const express = require("express");
2  const router = express.Router();
3  const artisanController = require("../controllers/artisan.controller");
4  const requireAuth = require("../middlewares/auth.middleware");
5
6  // --- Routes publiques (consultation) ---
7  router.get("/", artisanController.getAll);
8  router.get("/top", artisanController.getTop);
9  router.get("/categorie/:id", artisanController.getByCategorie);
10 router.get("/search/:nom", artisanController.search);
11 router.get("/:id", artisanController.getById);
12
13 // --- Routes admin (écriture), protégées par JWT ---
14 router.post("/", requireAuth, artisanController.create);
15 router.put("/:id", requireAuth, artisanController.update);
16 router.delete("/:id", requireAuth, artisanController.remove);
17
18 module.exports = router;
19

```

Le controller gère le cycle requête/réponse HTTP : il valide les champs obligatoires, appelle le service, puis traduit le résultat en réponse HTTP appropriée :

```

50 async function create(req, res) {
51   try {
52     const { nom, id_specialite } = req.body;
53     if (!nom || !id_specialite) {
54       return res
55         .status(400)
56         .json({
57           message: "Les champs 'nom' et 'id_specialite' sont obligatoires",
58         });
59     }
60     const artisan = await artisanService.createArtisan(req.body);
61     res.status(201).json(artisan);
62   } catch (error) {
63     res.status(500).json({ message: "Erreur serveur", error: error.message });
64   }
65 }

```

Le service concentre toute la logique métier et les accès Sequelize : c'est la seule couche qui manipule directement les modèles, indépendamment de la façon dont la requête a été reçue :

```

43 async function createArtisan(data) {
44   return Artisan.create(data);
45 }
46
47 async function updateArtisan(id, data) {
48   const artisan = await Artisan.findByPk(id);
49   if (!artisan) return null;
50   return artisan.update(data);
51 }

```

Cette séparation rend les responsabilités explicites : la route gère le routage, le controller gère le cycle HTTP et les validations simples, et le service réalise les opérations métier et l'accès aux

données. La même organisation est utilisée pour les artisans, les catégories, les spécialités et l'authentification.

4.7 Sécurité

La sécurisation du projet a été pensée à la fois côté front-end et côté back-end, avec une attention particulière portée à l'espace d'administration, seul point d'entrée permettant de modifier les données.

Sécurité côté front-end (React)

```

142   <input
143     type="email"
144     className="form-control"
145     value={form.email}
146     onChange={(e) =>
147       |   setForm({ ...form, email: e.target.value })
148     }
149     required
150   />

```

React échappe par défaut les valeurs insérées dans le JSX lorsqu'elles sont rendues comme du texte. Cela réduit notamment le risque de XSS lors de l'affichage de données provenant de l'API. Cette protection ne constitue toutefois pas une sécurité globale de toutes les entrées et ne remplace pas les contrôles côté serveur.

Les champs de saisie du formulaire de contact et des formulaires du back-office utilisent des composants contrôlés (value + onChange), associés à des contraintes HTML natives. Pour les opérations d'administration, des contrôles complémentaires existent dans les controllers côté serveur ; le formulaire de contact, lui, est envoyé directement à EmailJS depuis le front-end.

React échappe par défaut les valeurs insérées dans le JSX lorsqu'elles sont rendues comme du texte. Cela réduit notamment le risque de XSS lors de l'affichage de données provenant de l'API. Cette protection ne constitue toutefois pas une sécurité globale de toutes les entrées et ne remplace pas les contrôles côté serveur.

Les champs de saisie du formulaire de contact et des formulaires du back-office utilisent des composants contrôlés (value + onChange), associés à des contraintes HTML natives. Pour les opérations d'administration, des contrôles complémentaires existent dans les controllers côté serveur ; le formulaire de contact, lui, est envoyé directement à EmailJS depuis le front-end.

```

<input
  type="email"
  className="form-control"
  value={form.email}
  onChange={(e) => setForm({ ...form, email: e.target.value })}
  required
/>

```

L'attribut required empêche la soumission d'un champ vide et type="email" impose un format d'adresse électronique reconnu par le navigateur. Pour la note d'un artisan, type="number" combiné à min/max limite les valeurs saisies côté client. Ces contrôles améliorent l'expérience utilisateur, mais ne constituent pas à eux seuls une barrière de sécurité.

Sécurité côté back-end (Node/Express/Sequelize)

Extrait du middleware JWT et d'une opération Sequelize utilisée par l'API.

L'ORM Sequelize construit ses requêtes avec des paramètres liés (bindings) plutôt que par concaténation de chaînes de caractères. Ainsi, même une requête simple comme celle-ci est protégée nativement contre l'injection SQL :

```
Artisan.findAll({ where: { nom: { [Op.like]: `>${nom}%` } } });
```

Sequelize échappe automatiquement la valeur de nom avant de l'insérer dans la requête SQL finale. Cette protection ne s'applique plus si l'on utilise `sequelize.query()` en mode brut avec une chaîne concaténée à la main, une pratique volontairement évitée dans ce projet, où tous les accès aux données passent par les méthodes Sequelize sécurisées (`findAll`, `findByPk`, `create`, `update`, `destroy`).

L'accès à l'espace d'administration repose sur un jeton JWT (JSON Web Token), généré à la connexion et valable deux heures :

```
const token = jwt.sign(
  { id_admin: admin.id_admin, email: admin.email },
  process.env.JWT_SECRET,
  { expiresIn: "2h" },
);
```

Ce jeton doit être transmis dans l'en-tête `Authorization` de chaque requête vers une route protégée. Un middleware dédié vérifie sa validité avant d'autoriser l'accès :

```
1  const jwt = require("jsonwebtoken");
2
3  // Protège les routes admin : vérifie le token JWT envoyé dans l'en-tête Authorization
4  function requireAuth(req, res, next) {
5    const authHeader = req.headers.authorization; // format attendu : "Bearer <token>"
6
7    if (!authHeader || !authHeader.startsWith("Bearer ")) {
8      return res.status(401).json({ message: "Token manquant" });
9    }
10
11    const token = authHeader.split(" ")[1];
12
13    try {
14      const payload = jwt.verify(token, process.env.JWT_SECRET);
15      req.admin = payload; // { id_admin, email } disponible dans la suite de la requête
16      next();
17    } catch (error) {
18      return res.status(403).json({ message: "Token invalide ou expiré" });
19    }
20  }
21
22  module.exports = requireAuth;
23
```

Toutes les routes de création, modification et suppression (artisans, catégories, spécialités) sont protégées par ce middleware. Les routes de consultation restent, elles, publiques, conformément au fonctionnement attendu du site.

Le mot de passe administrateur n'est jamais stocké en clair : il est haché avec `bcrypt` avant son enregistrement en base. Lors de la connexion, `bcrypt.compare()` compare le mot de passe fourni au hash stocké sans reconstituer le mot de passe original.

```
const motDePasseHache = await bcrypt.hash(motDePasse, 10);
```

À la connexion, le mot de passe saisi est comparé au hash stocké via `bcrypt.compare()`, sans jamais reconstituer le mot de passe original. Une fonctionnalité de changement de mot de passe est également disponible depuis l'espace admin, avec vérification de l'ancien mot de passe avant toute modification.

Une route d'administration non exposée

L'espace d'administration est accessible via la route `/admin`, qui n'apparaît dans aucun menu de navigation du site public. L'interface cliente vérifie la présence d'un jeton avant d'afficher le contenu d'administration, mais cette vérification n'est pas suffisante seule : l'API vérifie elle-même le JWT côté serveur avant toute opération d'écriture.

4.8 Sémantique HTML et SEO

Pourquoi la sémantique HTML ?

La sémantique HTML consiste à choisir des balises qui décrivent le rôle du contenu plutôt que d'utiliser uniquement des conteneurs génériques comme `div`. Elle améliore la compréhension de la page par les navigateurs, les technologies d'assistance et les moteurs de recherche.

La structure sémantique du projet

```

1  import React, { useState, useEffect } from "react";
2  import { Link, useNavigate } from "react-router-dom";
3  import axios from "axios";
4
5  const API_URL = process.env.REACT_APP_API_URL || "http://localhost:4000/api";
6
7  function Header() {
8    const [categories, setCategories] = useState([]);
9    const [search, setSearch] = useState("");
10   const navigate = useNavigate();
11
12   useEffect(() => {
13     axios
14       .get(`${API_URL}/categories`)
15       .then((res) => setCategories(Array.isArray(res.data) ? res.data : []))
16       .catch((err) => console.error(err));
17   }, []);
18
19   const handleSearch = (e) => {
20     e.preventDefault();
21     if (search.trim()) {
22       navigate(`/search/${search}`);
23     }
24   };
25
26   return (
27     <header className="navbar navbar-expand-lg px-0">
28       <div className="container">
29         <Link className="navbar-brand me-4" to="/">
30           
31         </Link>
32         <button
33           className="navbar-toggler"

```

Dans ce projet, le composant Header utilise une balise `<header>` contenant une navigation `<nav>`, structurée avec une liste `<ul>` et des éléments `<li>`. Les pages principales utilisent `<main>` et des `<section>` pour organiser les grandes zones de contenu. Le composant Footer utilise `<footer>`. Ces balises permettent d'identifier clairement les régions de la page.

Hiérarchie des titres

Les pages utilisent des titres h1 pour leur titre principal, puis des h2 et h3 pour les sous-parties. Une hiérarchie cohérente facilite la lecture du contenu et sa navigation par les technologies d'assistance.

Sémantique et SEO

La structure sémantique contribue au référencement en donnant du contexte au contenu. Elle complète d'autres éléments SEO : title, meta description, URLs lisibles, textes alternatifs des images et qualité du contenu.

Éléments effectivement présents et limites

Le fichier `public/index.html` contient une balise `title` et une meta description adaptée au site. La langue du document est déclarée en français avec `lang="fr"`. Ces éléments contribuent à améliorer l'identification et le référencement du site auprès des moteurs de recherche..

Rendu côté client

Le contenu des artisans et des catégories est chargé après exécution du JavaScript par React. Le projet n'utilise pas de rendu serveur : une évolution vers du rendu côté serveur ou de la génération statique améliorerait l'exposition du contenu dynamique aux moteurs de recherche.

Preuve par le code

```

1  <!doctype html>
2  <html lang="fr">
3    <head>
4      <meta charset="utf-8" />
5      <link rel="icon" href="%PUBLIC_URL%/favicon-32.ico" />
6      <meta name="viewport" content="width=device-width, initial-scale=1" />
7      <meta name="theme-color" content="#000000" />
8      <meta
9        name="description"
10       content="Trouve ton artisan ! Trouvez facilement un artisan près de chez vous en A
11     />
12     <link rel="apple-touch-icon" href="%PUBLIC_URL%/logo192.png" />
13     <!--
14       manifest.json provides metadata used when your web app is installed on a
15       user's mobile device or desktop. See https://developers.google.com/web/fundamenta
16     -->
17     <link rel="manifest" href="%PUBLIC_URL%/manifest.json" />
18     <!--
19       Notice the use of %PUBLIC_URL% in the tags above.
20       It will be replaced with the URL of the `public` folder during the build.
21       Only files inside the `public` folder can be referenced from the HTML.
22
23       Unlike "/favicon.ico" or "favicon.ico", "%PUBLIC_URL%/favicon.ico" will
24       work correctly both with client-side routing and a non-root public URL.
25       Learn how to configure a non-root public URL by running `npm run build`.
26     -->
27     <title>Trouve ton artisan !</title>
28   </head>
29   <body>
30     <noscript>You need to enable JavaScript to run this app.</noscript>
31     <div id="root"></div>

```

4.9 Accessibilité et inclusion

Plusieurs choix de conception sur "Trouve ton artisan" contribuent à une meilleure accessibilité de l'interface :

- **Contraste et lisibilité** : le choix d'un texte foncé sur fond clair (et inversement pour les bandeaux colorés) a été pensé pour rester suffisamment contrasté et lisible, un critère d'accessibilité à part entière au-delà du simple rendu esthétique
- **Attributs sémantiques** : les images du site (logo, illustrations de fiches artisan) sont accompagnées d'attributs alt décrivant leur contenu, permettant leur interprétation par les lecteurs d'écran
- **Structure sémantique** : les balises significatives utilisées dans les composants sont détaillées dans la partie 4.8 ; elles facilitent notamment la compréhension de la structure par les technologies d'assistance.
- **Responsive design** : l'adaptation du site aux différentes tailles d'écran (partie 4.2) profite également aux utilisateurs de technologies d'assistance mobiles

Méthodologie et organisation du travail

Démarche de conception

Le développement de "Trouve ton artisan" a suivi une démarche séquentielle classique en développement web : conception des maquettes sur Figma dans un premier temps, validation de la structure de navigation et de l'identité visuelle, puis développement en parallèle du back-end (modélisation de la base, API) et du front-end (intégration statique, puis dynamisation). Cette approche a permis de disposer d'une API fonctionnelle avant même de commencer l'intégration des appels dynamiques côté React, réduisant les allers-retours entre les deux couches de l'application.

Gestion des versions avec Git

L'ensemble du code du projet est versionné avec Git et hébergé sur GitHub. Chaque modification significative a fait l'objet d'un commit dédié, avec un message décrivant clairement la nature du changement (par exemple « Ajout de l'envoi d'email via EmailJS pour le formulaire de contact », ou « Correction du style de la barre de navigation »), facilitant la traçabilité des évolutions du projet dans le temps. Cette discipline de commit s'est révélée particulièrement utile lors du débogage : pouvoir identifier précisément à quel moment une fonctionnalité a été introduite ou modifiée a permis de cibler plus rapidement l'origine de certains bugs rencontrés en cours de projet.

Itération et tests manuels

Faute d'avoir mis en place une suite de tests automatisés sur ce projet, la validation du bon fonctionnement s'est faite par des tests manuels répétés à chaque évolution significative : rechargement du site après chaque modification de code, vérification visuelle du rendu sur différentes tailles d'écran, test systématique des parcours utilisateurs principaux après un déploiement. Cette approche, bien que moins rigoureuse qu'une suite de tests automatisés, a permis de détecter rapidement des régressions, comme lors de l'intégration d'EmailJS où plusieurs allers-retours entre le code et le navigateur ont été nécessaires pour isoler un bug de copier-coller récurrent (détaillé en partie « Difficultés rencontrées et solutions »).

Outils de suivi

Le suivi de l'avancement du projet s'est fait de façon simple, adaptée à un travail individuel : une liste de tâches restant à accomplir était tenue à jour au fil de l'avancement (finalisation du back-end, peuplement de la base de données, correction des bugs d'affichage, intégration de l'envoi d'email réel), permettant de prioriser les actions restantes à chaque session de travail plutôt que de disperser les efforts sur plusieurs fronts simultanément.

Schéma de navigation

Le schéma ci-dessous représente l'ensemble des parcours possibles au sein du site "Trouve ton artisan", depuis la page d'accueil jusqu'aux différentes pages de destination. Il illustre concrètement l'architecture de routing mise en place avec React Router, détaillée en partie 4.3.

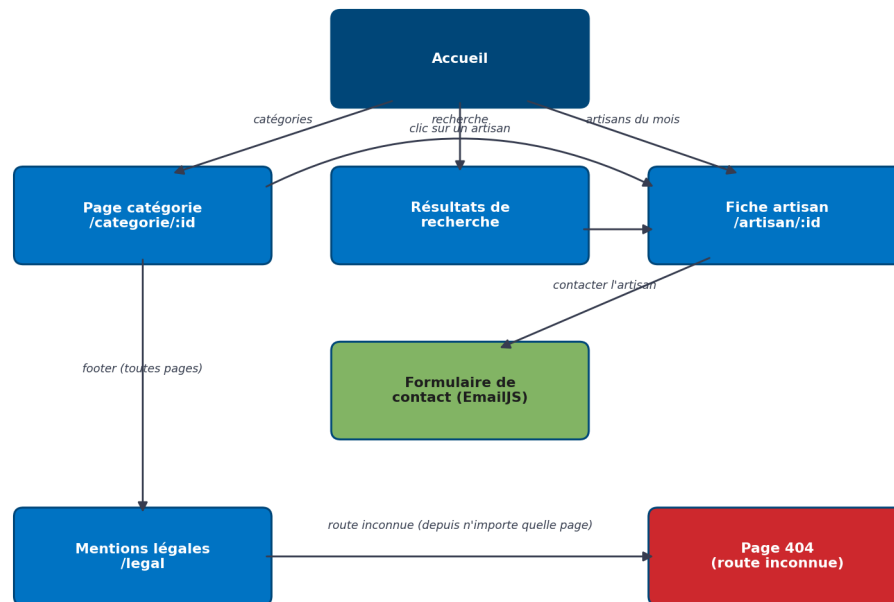


Schéma de navigation du site "Trouve ton artisan"

Trois parcours partent de la page d'accueil : le clic sur une catégorie du menu, la recherche par nom, et le clic sur un artisan mis en avant ("artisans du mois"). Ces trois chemins convergent tous, à terme, vers la fiche détaillée d'un artisan, qui constitue la page pivot du site : c'est depuis cette page que l'utilisateur accède au formulaire de contact, objectif final du parcours utilisateur. Le pied de page, commun à toutes les pages, donne un accès permanent à la page "Mentions légales". Enfin, toute tentative d'accès à une URL non gérée par l'application redirige systématiquement vers la page 404 personnalisée.

Jeu d'essai représentatif

Objectifs du test

Ce jeu d'essai vise à vérifier le bon fonctionnement des principales fonctionnalités du site "Trouve ton artisan" en conditions réelles d'utilisation : navigation, recherche, envoi du formulaire de contact, gestion des erreurs, et comportement face aux limites des hébergements gratuits utilisés pour ce projet.

Résultats des tests

N°	Test réalisé	Donnée en entrée	Résultat attendu	Résultat obtenu	Écart
1	Chargement de la liste des catégories au montage de la page d'accueil	Appel GET /api/categories	Affichage des 4 catégories (Bâtiment, Services, Fabrication, Alimentation) dans le menu	Les 4 catégories s'affichent correctement	Aucun
2	Navigation vers une page catégorie	Clic sur "Bâtiment"	Affichage des artisans de la catégorie Bâtiment avec leur spécialité	4 artisans affichés (Orville Salmons, Boutot & fils, Mont Blanc Électricité, Vallis Bellemare)	Aucun
3	Recherche par nom	Saisie de "orville" dans la barre de recherche	Redirection vers une page de résultats affichant les artisans correspondants	1 artisan trouvé (Orville Salmons), affiché correctement	Aucun
4	Envoi du formulaire de contact	Remplissage des 4 champs (nom, email, objet, message) et clic sur "Envoyer"	Email reçu sur l'adresse de l'artisan, message de confirmation affiché	Email bien reçu via EmailJS, message de confirmation affiché	Aucun (après correction, cf. partie difficultés)
5	Accès à une URL inexistante	Saisie d'une URL non définie dans le routing (ex. /nimportequoi)	Affichage de la page 404 personnalisée avec bouton de retour à l'accueil	Page 404 affichée correctement	Aucun
6	Interruption de la base de données (Aiven en veille)	Appel à l'API après une longue période d'inactivité	L'API doit répondre normalement une fois les services réveillés	Erreur getaddrinfo ENOTFOUND tant qu'Aiven n'est pas manuellement redémarré	Écart : nécessite une action manuelle, documenté en partie difficultés

Jeu d'essai — Back-office

Ce second jeu d'essai vise à valider spécifiquement l'espace d'administration : authentification, protection des routes d'écriture, et cycle CRUD complet sur la ressource artisan.

N°	Test réalisé	Donnée en entrée	Résultat attendu	Résultat obtenu	Écart
7	Connexion à l'espace admin avec identifiants valides	POST /api/admin/login avec email + mot de passe corrects	Réception d'un jeton JWT, redirection vers /admin	Jeton reçu, redirection effective vers le tableau de bord admin	Aucun
8	Connexion avec mot de passe incorrect	POST /api/admin/login avec un mauvais mot de passe	Rejet avec statut 401, message « Identifiants invalides »	Statut 401 reçu, message affiché sur le formulaire	Aucun
9	Accès à une route d'écriture sans jeton	POST /api/artisans sans en-tête Authorization	Rejet avec statut 401, message « Token manquant »	Statut 401 reçu comme attendu	Aucun
10	Création d'un artisan depuis le back-office	Formulaire rempli (nom, spécialité, ville) et validation	Nouvel artisan visible dans le tableau admin et sur le site public	Artisan créé, visible immédiatement dans les deux vues	Aucun
11	Modification d'un artisan existant	Clic sur « Modifier », changement de la note, enregistrement	Nouvelle note affichée dans le tableau et sur la fiche publique	Mise à jour effective, confirmée sur les deux vues	Aucun
12	Suppression d'un artisan	Clic sur « Supprimer » sur un artisan test	L'artisan disparaît du tableau admin et du site public	Suppression confirmée, artisan absent des deux vues	Aucun

L'ensemble de ces tests n'a révélé aucun écart : l'authentification fonctionne comme attendu, les routes d'écriture sont correctement protégées contre les requêtes non authentifiées, et le cycle CRUD complet (création, modification, suppression) se répercute immédiatement à la fois dans le tableau d'administration et sur le site public, confirmant que les deux interfaces exploitent bien la même base de données.

Analyse des écarts

Cinq des six tests réalisés n'ont révélé aucun écart entre le résultat attendu et le résultat obtenu, confirmant le bon fonctionnement du site sur ses parcours principaux. Le sixième test (interruption de la base de données) a révélé un écart lié aux limites des hébergements gratuits utilisés (Render et Aiven), qui appliquent une mise en veille après une période d'inactivité. Cet écart ne remet pas en cause le fonctionnement du code lui-même, mais constitue une contrainte d'infrastructure à anticiper avant toute démonstration en direct — ce point est développé dans la partie « Difficultés rencontrées et solutions ».

Conclusion du jeu d'essai

L'ensemble des fonctionnalités testées répond aux besoins fonctionnels définis en partie 2 (Expression des besoins). Le seul point de vigilance identifié concerne la disponibilité des services d'hébergement gratuits, pour lequel une procédure de vérification préalable a été mise en place avant toute présentation du projet.

Stratégie de validation

La validation du projet a été réalisée progressivement, au fur et à mesure de l'implémentation des fonctionnalités. Chaque ajout important a été vérifié dans le navigateur et, lorsque cela concernait l'API, à partir des réponses HTTP et des journaux disponibles. Cette démarche a permis de détecter les régressions avant de poursuivre le développement.

Tests fonctionnels du site public

Les parcours principaux ont été testés en suivant le comportement attendu d'un visiteur : chargement des catégories, navigation vers une catégorie, consultation d'une fiche artisan, recherche par nom et utilisation du formulaire de contact.

Pour chaque fonctionnalité, j'ai comparé le résultat obtenu avec le résultat attendu. Les cas nominaux ont été complétés par des situations d'erreur, par exemple une recherche ne retournant aucun artisan ou l'accès à une URL qui n'existe pas afin de vérifier l'affichage de la page 404.

Tests du BackOffice et de la sécurité

Le BackOffice a fait l'objet de vérifications spécifiques : connexion avec des identifiants valides, refus d'accès avec des identifiants invalides, accès aux routes d'écriture sans jeton, puis tests du cycle CRUD sur les artisans. J'ai également vérifié que les modifications réalisées dans l'administration étaient visibles sur le site public.

Ces tests permettent de vérifier que la protection ne repose pas uniquement sur l'interface. Même si un utilisateur connaît l'URL du BackOffice, les opérations sensibles restent soumises au contrôle d'authentification côté serveur.

Tests responsive

L'interface a été vérifiée sur plusieurs largeurs d'écran. Les tests portent notamment sur le comportement du menu, la taille des titres, l'organisation des étapes de navigation et le nombre de colonnes utilisées pour l'affichage des cartes artisans. Les captures desktop, tablette et mobile présentées dans la partie 4.2 illustrent ces vérifications.

Veille technologique et sécurité

Méthodologie de veille

Tout au long du développement de ce projet, une veille informelle a été menée à travers la documentation officielle des technologies utilisées (React, Express, Sequelize, JWT, bcrypt), ainsi que par l'observation directe des journaux de déploiement (logs Render, Aiven, Netlify), qui ont permis d'identifier plusieurs points d'amélioration au fil du projet.

Vulnérabilités et points d'attention identifiés

- Exposition potentielle d'identifiants sensibles : risque de versionner par erreur les identifiants de connexion à la base de données ou les clés d'API sur GitHub
- Absence de limitation de débit (rate limiting) sur les routes de l'API, qui pourrait exposer le service à des appels abusifs
- Absence de validation stricte des entrées côté serveur (les champs sont validés côté front-end, mais pas systématiquement revérifiés côté API)

- Dépendances npm : la commande npm audit a signalé plusieurs vulnérabilités mineures à modérées dans des dépendances indirectes du projet front-end

Correctifs et bonnes pratiques mises en place

- Utilisation systématique de variables d'environnement pour les identifiants sensibles, associées à un fichier .gitignore excluant les fichiers .env du versioning
- Mise en place d'un middleware d'authentification JWT sur l'espace d'administration de "Trouve ton artisan" pour restreindre l'accès aux routes d'écriture (détaillé en partie 4.6)
- Encapsulation systématique des opérations de base de données dans des blocs try/catch, limitant l'exposition d'informations techniques en cas d'erreur

Résultats de la veille

Cette veille a permis d'identifier des axes d'amélioration réalistes pour la suite du projet, en particulier l'ajout d'une validation des entrées côté serveur (actuellement partielle) et la mise en place d'un rate limiting, qui n'ont pas pu être développés dans le temps imparti pour ce devoir mais qui constituent des pistes concrètes d'évolution.

Conclusion

Cette démarche de veille, bien que menée de façon informelle sur un projet de formation, illustre une posture professionnelle essentielle en développement web : rester attentif aux limites de sécurité de ce que l'on construit, même lorsque le contexte (projet pédagogique, données fictives) ne l'exige pas strictement.

Bilan technique global

Correspondance entre les réalisations et les compétences

Le projet final permet de présenter une continuité entre la conception, l'intégration, la dynamique du front-end et les composants du back-end. L'objectif n'est pas seulement de présenter les technologies utilisées, mais de montrer les réalisations concrètes qui répondent aux six compétences professionnelles mobilisées dans le projet.

Maquetter des interfaces utilisateur web ou web mobile — les maquettes Figma définissent les pages principales et les parcours de navigation.

Réaliser des interfaces utilisateur statiques web ou web mobile — l'intégration React, Bootstrap et Sass est adaptée aux différents supports, avec les captures responsive comme preuve visuelle.

Développer la partie dynamique des interfaces utilisateur web ou web mobile — les composants React récupèrent les données, utilisent le routing, réalisent la recherche et gèrent le formulaire de contact.

Mettre en place une base de données relationnelle — la modélisation Merise et la base MySQL organisent les catégories, spécialités et artisans au moyen de clés et de relations.

Développer des composants d'accès aux données SQL — le projet met en œuvre l'accès aux données SQL avec Sequelize et les modèles associés ; aucun composant NoSQL n'a été ajouté artificiellement.

Développer des composants métier côté serveur — l'API Express, la logique métier et la séparation route/controller/service sont notamment illustrées par le BackOffice.

Cette correspondance montre pourquoi le projet peut être présenté comme un projet full-stack unique : les fonctionnalités du site public et du BackOffice s'appuient sur la même architecture et sur la même base de données.

Avant de détailler les difficultés rencontrées, ce bilan synthétise la manière dont le site public et l'espace d'administration se complètent pour couvrir l'ensemble des compétences attendues, en confrontant deux approches techniques différentes sur des problématiques comparables.

Une architecture complète sur un seul projet

"Trouve ton artisan" couvre à lui seul l'ensemble des six compétences professionnelles mobilisées dans le projet du référentiel, grâce à l'ajout d'un espace d'administration (back-office) venant compléter la partie front-end publique déjà en place : la même base de données MySQL est désormais exploitée à la fois en lecture (site public) et en écriture protégée (back-office), et la même architecture route / controller / service structure l'ensemble des composants métier côté serveur, qu'ils soient publics ou réservés à l'administrateur.

Deux niveaux d'exigence en matière de sécurité, sur un même projet

Le contraste entre les routes de consultation, ouvertes à tous, et les routes d'écriture du back-office, protégées par authentification JWT et mot de passe haché, a permis d'expérimenter deux postures de sécurité différentes au sein d'un seul et même projet : une simple robustesse face aux erreurs côté public, et un véritable contrôle d'accès côté administration.

Synthèse

Ce bilan met en évidence que le projet "Trouve ton artisan", une fois complété par son espace d'administration, offre une vue d'ensemble cohérente des compétences du Titre Professionnel Développeur Web et Web Mobile, sans qu'il soit nécessaire de le combiner à un second projet pour en démontrer la maîtrise complète.

5. Difficultés rencontrées et solutions

Migration de la base de données (Railway → Aiven)

Contexte et symptômes

La base MySQL du projet "Trouve ton artisan" était initialement hébergée sur Railway. Suite à l'expiration de la période d'essai gratuite, la base est devenue inaccessible, provoquant des erreurs 503 en cascade sur l'API déployée sur Render, elles-mêmes répercutées sous forme d'erreurs JavaScript côté front-end (échecs d'appels `.map()` sur des données non reçues, dans les composants Header, Accueil et FicheArtisan).

Solution mise en œuvre

J'ai résolu ce problème en migrant l'intégralité de la base vers Aiven, un autre hébergeur MySQL managé proposant une offre gratuite pérenne : recréation du schéma (categorie, specialite, artisan) via le script SQL détaillé en partie 4.4, réinsertion des données de démonstration, puis mise à jour des variables d'environnement de l'API sur Render pour pointer vers la nouvelle base. En parallèle, j'ai sécurisé le code front-end contre ce type d'incident en ajoutant des vérifications `Array.isArray` avant les appels `.map()`, afin que le site affiche une liste vide plutôt que de planter si l'API venait à nouveau à ne pas répondre correctement.

Compétences mobilisées

- Diagnostic d'une chaîne d'erreurs en remontant de l'affichage front-end jusqu'à la cause racine côté base de données
- Migration de schéma et de données entre deux hébergeurs MySQL différents
- Écriture de code défensif (vérifications de type) pour limiter l'impact d'une indisponibilité de service

Mise en veille des hébergements gratuits (Render et Aiven)

Contexte et symptômes

Les deux services d'hébergement gratuits utilisés dans ce projet (Render pour les API, Aiven pour la base MySQL) appliquent une mise en veille automatique après une période d'inactivité, avec des comportements différents : Render redémarre automatiquement dès qu'une requête lui parvient (délai de 30 à 60 secondes), tandis qu'Aiven bascule le service MySQL en état « Powered off », nécessitant une réactivation manuelle.

Solution mise en œuvre

J'ai identifié cette différence de comportement en observant une erreur de connexion (`getaddrinfo ENOTFOUND`) dans les journaux de déploiement Render, qui m'a conduit à vérifier l'état du service Aiven depuis sa console d'administration, puis à le redémarrer manuellement via le bouton de réactivation prévu à cet effet. Pour éviter que cet incident ne se reproduise au moment le plus critique, j'ai mis en place une vérification systématique de la disponibilité des deux services quelques minutes avant toute démonstration du projet.

Compétences mobilisées

- Lecture et interprétation de journaux de déploiement (logs) pour diagnostiquer une erreur réseau
- Compréhension des différences de comportement entre plusieurs offres d'hébergement gratuites
- Anticipation d'un risque opérationnel par la mise en place d'une procédure de vérification préalable

Conflit Git bloqué en plein merge (Vim)

Contexte et symptômes

Lors de la correction d'un bug sur "Trouve ton artisan", un git merge a généré un conflit qui a ouvert l'éditeur Vim en ligne de commande — un outil que je ne maîtrisais pas à ce moment-là. Je me suis retrouvée bloquée dans l'éditeur sans savoir comment en sortir ni valider le message de merge, le terminal semblant figé.

Solution mise en œuvre

Après recherche, j'ai appris à résoudre ce type de situation en tapant la séquence :wq (write and quit) pour sauvegarder le message de merge par défaut et quitter Vim. Pour éviter de me retrouver à nouveau bloquée de cette façon, j'ai ensuite configuré un éditeur par défaut plus familier pour Git, via la commande `git config --global core.editor`, qui ouvre désormais VS Code en cas de besoin de rédiger un message de commit ou de merge en ligne de commande.

Compétences mobilisées

- Recherche autonome de solution face à un blocage inconnu en ligne de commande
- Compréhension du fonctionnement d'un éditeur de texte modal (Vim) et de ses commandes de base
- Configuration durable de l'environnement Git pour prévenir la récurrence du problème

Bug de compilation récurrent lors de l'intégration d'EmailJS

Contexte et symptômes

Lors de l'ajout de l'envoi d'email réel au formulaire de contact (remplacement d'une simulation par un appel à l'API EmailJS), plusieurs tentatives de copier-coller de code ont provoqué une erreur de syntaxe récurrente dans VS Code : la balise `<a>` d'un lien disparaissait systématiquement lors du collage, provoquant une erreur de compilation Babel (« Unexpected token, expected "," ») à chaque tentative de correction par un nouveau copier-coller du même bloc.

Solution mise en œuvre

J'ai identifié la cause probable (un conflit entre le presse-papiers et l'autocomplétion automatique des chevrons dans VS Code) après avoir constaté que l'erreur se reproduisait à l'identique malgré plusieurs tentatives de collage du code corrigé. J'ai contourné le problème en retapant manuellement, caractère par caractère, la portion de code concernée plutôt qu'en la collant, ce qui a permis de compiler sans erreur. La même méthode a ensuite été appliquée avec succès pour corriger l'identifiant de template EmailJS, qui s'est révélé incorrect lors des premiers tests d'envoi.

Compétences mobilisées

- Isolation méthodique d'une erreur récurrente en changeant une seule variable à la fois (le mode de saisie du code) plutôt qu'en répétant la même action
- Lecture et interprétation d'un message d'erreur de compilation Babel/Webpack pour localiser précisément la ligne fautive
- Persévérance face à un bug résistant à plusieurs tentatives de correction successives

Conclusion

Le projet présenté dans ce dossier, "Trouve ton artisan", m'a permis de mettre en œuvre l'ensemble des compétences attendues pour le Titre Professionnel Développeur Web et Web Mobile : conception d'interfaces à partir de maquettes, développement front-end réactif avec React, mise en place et exploitation d'une base de données relationnelle MySQL, et développement de composants métier sécurisés côté serveur, aussi bien pour le site public que pour l'espace d'administration.

Le moment que j'ai le plus apprécié dans ce projet a été de voir le passage de la maquette Figma au résultat final concret : constater qu'une interface pensée sur papier prenne vie et fonctionne réellement reste, pour moi, la partie la plus gratifiante du développement web. À l'inverse, les phases de débogage ont été les plus éprouvantes, notamment lorsqu'un problème récurrent nécessitait d'isoler patiemment sa cause avant de pouvoir le résoudre — un exercice de patience que je continue de travailler.

Au-delà des compétences techniques évaluées par ce dossier, la réalisation de ce projet m'a permis de développer des compétences transversales tout aussi essentielles au métier de développeuse web : la capacité à diagnostiquer méthodiquement un problème en remontant de ses symptômes à sa cause, la rigueur dans la gestion des versions et le suivi des modifications apportées au code, et l'autonomie nécessaire pour rechercher et mettre en œuvre une solution face à un blocage technique inconnu.

Cette formation et la réalisation de ces projets constituent une base solide pour aborder mon entrée dans le métier de développeuse web, avec l'envie de continuer à approfondir mes connaissances, notamment sur les notions théoriques de modélisation de données et d'architecture back-end, ainsi que sur la mise en place de tests automatisés, qui n'ont pas pu être développés dans le cadre de ce projet faute de temps mais qui constituent une prochaine étape naturelle dans ma progression professionnelle.

Perspectives d'évolution

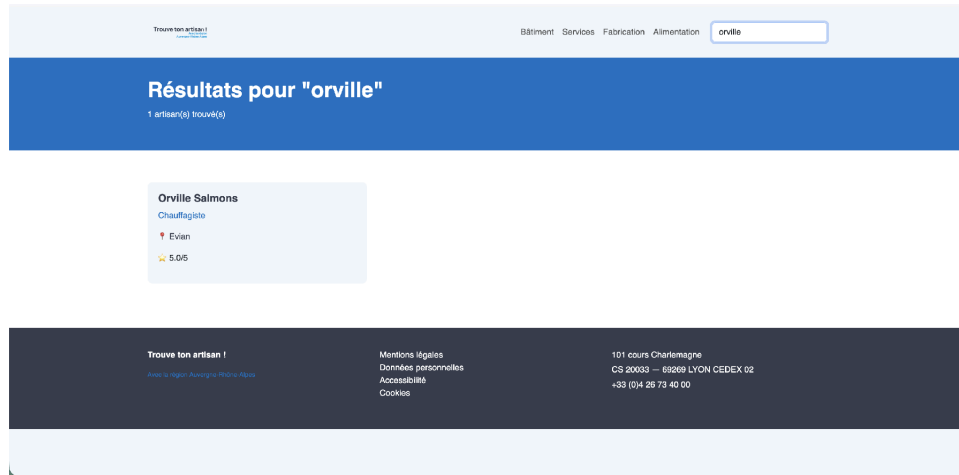
Plusieurs pistes d'amélioration ont été identifiées au fil de la rédaction de ce dossier, sans avoir pu être développées dans le temps imparti pour ce devoir : l'ouverture d'un accès d'édition restreint aux artisans eux-mêmes sur leur propre fiche, en complément de l'espace d'administration global désormais en place ; l'ajout d'une suite de tests automatisés, actuellement validée par des tests manuels uniquement (cf. partie « Jeu d'essai représentatif ») ; la mise en place d'un rendu côté serveur (Next.js) pour améliorer le référencement du site, évoquée en partie 4.8 ; et la mise en place d'un mécanisme de ping automatique périodique pour éviter la mise en veille des services d'hébergement gratuits, contrainte détaillée dans la partie « Difficultés rencontrées et solutions ».

Sur le plan personnel, ce dossier a également été l'occasion de mesurer le chemin parcouru depuis le début de la formation : la capacité à construire une architecture complète sur une seule stack technique (React/Node/MySQL), en étendant un projet existant par un nouvel espace d'administration sécurisé plutôt qu'en le juxtaposant à un second projet, et à documenter précisément des choix techniques et des difficultés rencontrées, reflète une autonomie qui n'était pas acquise en début de parcours.

Annexes

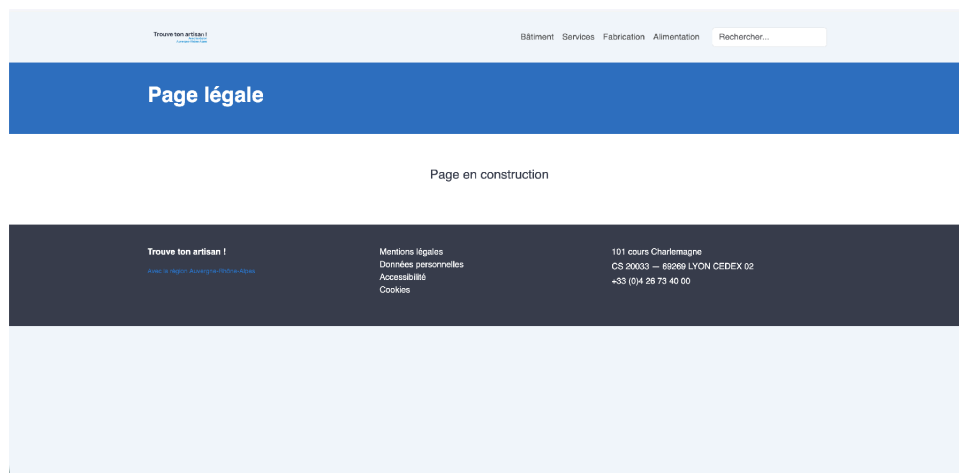
Annexe 1 — Captures d'écran complémentaires

Résultat de la fonctionnalité de recherche dynamique, illustrant la compétence 4.3 :

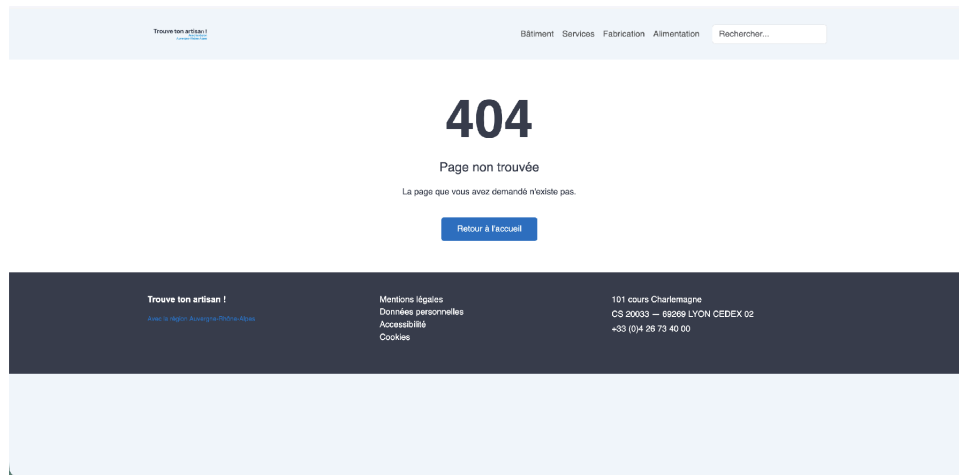


Page de résultats de recherche pour le terme "orville"

La page « Mentions légales » reprend fidèlement le contenu prévu dès la maquette Figma (« Page en construction »), le contenu juridique détaillé n'étant pas l'objet de ce devoir.



Page "Mentions légales"



Page 404 personnalisée, avec retour à l'accueil

Annexe 2 — Modèles Sequelize complets (Trouve ton artisan)

Categorie.js

```

1  const { DataTypes } = require("sequelize");
2  const sequelize = require("../config/database");
3
4  const Categorie = sequelize.define(
5    "Categorie",
6    {
7      id_categorie: {
8        type: DataTypes.INTEGER,
9        primaryKey: true,
10       autoIncrement: true,
11      },
12      nom: {
13        type: DataTypes.STRING(100),
14        allowNull: false,
15      },
16    },
17    {
18      tableName: "categorie",
19      timestamps: false,
20    },
21  );
22
23  module.exports = Categorie;

```

Specialite.js

```

1  const { DataTypes } = require("sequelize");
2  const sequelize = require("../config/database");
3
4  const Specialite = sequelize.define(
5    "Specialite",
6    {
7      id_specialite: {
8        type: DataTypes.INTEGER,
9        primaryKey: true,
10       autoIncrement: true,
11      },
12      nom: {
13        type: DataTypes.STRING(100),
14        allowNull: false,
15      },
16      id_categorie: {
17        type: DataTypes.INTEGER,
18        allowNull: false,
19      },
20    },
21    {
22      tableName: "specialite",
23      timestamps: false,
24    },
25  );
26
27  module.exports = Specialite;

```

Annexe 3 — Routes API complètes (Trouve ton artisan)

Route des catégories

```
const express = require("express");
const router = express.Router();
const Categorie = require("../models/Categorie");

// GET toutes les catégories
router.get("/", async (req, res) => {
  try {
    const categories = await Categorie.findAll();
    res.json(categories);
  } catch (error) {
    res.status(500).json({ message: "Erreur serveur", error });
  }
});

module.exports = router;
```

Cette route illustre la forme la plus simple d'un composant métier côté serveur : une opération de lecture complète (findAll) sans filtre ni relation, utilisée pour alimenter le menu de navigation du site depuis n'importe quelle page.

Route complète des artisans (rappel, cf. partie 4.6)

```
const express = require("express");
const router = express.Router();
const Artisan = require("../models/Artisan");
const Specialite = require("../models/Specialite");
const Categorie = require("../models/Categorie");

// GET tous les artisans
router.get("/", async (req, res) => {
  try {
    const artisans = await Artisan.findAll({
      include: {
        model: Specialite,
        include: { model: Categorie },
      },
    });
    res.json(artisans);
  } catch (error) {
    res.status(500).json({ message: "Erreur serveur", error });
  }
});

// GET artisans du mois (top = true)
router.get("/top", async (req, res) => {
  try {
    const artisans = await Artisan.findAll({
      where: { top: true },
      include: {
        model: Specialite,
        include: { model: Categorie },
      },
    });
    res.json(artisans);
  } catch (error) {
    res.status(500).json({ message: "Erreur serveur", error });
  }
});

// GET artisans par catégorie
router.get("/categorie/:id", async (req, res) => {
  try {
    const artisans = await Artisan.findAll({
```

```
        include: {
          model: Specialite,
          where: { id_categorie: req.params.id },
          include: { model: Categorie },
        },
      });
      res.json(artisans);
    } catch (error) {
      res.status(500).json({ message: "Erreur serveur", error });
    }
  });
}

module.exports = router;
```

Annexe 4 — Liens du projet

- Site "Trouve ton artisan" : frabjous-cajeta-43c369.netlify.app
- API "Trouve ton artisan" : trouve-ton-artisan-api-0gng.onrender.com
- Dépôt GitHub front-end : github.com/sarahbrh/trouve-ton-artisan-front
- Dépôt GitHub API : github.com/sarahbrh/trouve-ton-artisan-api
- Espace administrateur (back-office) : accessible sur /admin depuis le site "Trouve ton artisan", non référencé dans la navigation publique

Remerciements

Je tiens à remercier ma coach Delphine, ainsi que les différents mentors avec qui j'ai pu échanger tout au long de ce projet, en particulier Alain, avec qui les échanges ont été les plus nombreux. Je remercie également l'équipe pédagogique du Centre Européen de Formation pour son accompagnement tout au long de cette formation.